



Skyline: a Rapid Prototyping Driving Simulator for User Experience

Ignacio Alvarez

Intel Labs
Hillsboro, OR
ignacio.j.alvarez@intel.com

Laura Rumbel

Intel SMG
Hillsboro, OR
laura.m.rumbel@intel.com

Robert Adams

Intel Labs
Hillsboro, OR
robert.adams@intel.com

ABSTRACT

This paper introduces Skyline, a user experience prototyping platform for automotive, developed in Intel Labs to enable rapid iterative development of in-vehicle experiences. The paper describes the hardware and software components of Skyline. It highlights the flexibility of the interior HMI configuration and the accessibility of the development platform, based on open source Web technologies such as JavaScript, CSS, Node.js and MQTT. The paper steps through the development and user testing processes for a cockpit experience with Skyline, illustrating the benefits of capturing early qualitative user feedback as support for rapid prototyping. Finally, the paper outlines the potential benefits of high fidelity assets developed on the platform for both industry and academia, and the enormous value that documented user experience HMI assets can have for in-vehicle feature productization and research.

Author Keywords

Skyline; Driving Simulator; Rapid Prototyping; User Experience; Design Thinking

ACM Classification Keywords

H.5.m. Information interfaces and presentation: Miscellaneous; Design, Experimentation, Human Factors

INTRODUCTION

Driving simulators are one of the most essential tools for researchers working in the areas of traffic safety and in-vehicle technology development and evaluation. As a research tool, their existence and use can be traced back to 1934 when Miles and Vincent crafted a device that produced an illusion of driving to address the need of compulsory tests for drivers, originally mandated as result of high road traffic deaths and injuries in the UK [1]. Thus, driving simulators were born as advanced training instruments to provide realistic experiences under controlled circumstances, and their adoption was often tied

to the increasing rate of traffic accidents, such as in the case of Japan in the 1960s [2].

Driving simulators have since then extended their usages from traffic safety research and training education to the evaluation of vehicle designs, infrastructure, and technology. Automakers have also put their own efforts into driving simulators to research the stability and maneuverability of novel car models before they reach customer's hands [1].

Realism and user experience (UX) in driving simulators have also benefited from continuous advances of immersive visual display technology. Eventually, with the widespread adoption and commercialization of realistic 3D virtual environments, driving simulators also became end consumer products in the form of driving games [3, 4].

Nowadays researchers and industry practitioners have at their disposition a wide range of simulator configurations ranging from low to high fidelity, in terms of the degree to which they reproduce real world situations to the visual, kinesthetic and auditory senses [5].

Depending on the development stage, the fidelity requirements, and the overall budget available, researchers may choose to run studies across diverse driving simulators from one extreme to another: having paper prototyped in-vehicle experiences, versus advance driving simulators capable of generating acceleration and torque motion feedback. The benefits of each extreme have been widely discussed [6,7,8,9]. On one end, low-level simulators enable rapid development and scarce investments and typically suffice to provide early feedback to designers. On the other side, advanced driver simulators require significant investments, but in return provide the highest accuracy on driving performance metrics, including subjective metrics [5].

The majority of researchers compromise with mid-fidelity driving simulators specially those which support open access software. In the recent years there has been a trend in favor of open source software for simulators such as the Lane Change Task [10] and OpenDS [11], provided that they grant access to the collection of standardized measures on mental workload, as well as physiological, subjective and performance metrics collected during the driving tasks advocated by Green in [12] and [13].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org. Copyright is held by the owner/author(s). Publication rights licensed to ACM.

AutomotiveUI '15, September 01 - 03, 2015, Nottingham, United Kingdom
© 2015 ACM. ISBN 978-1-4503-3736-6/15/09...\$15.00
DOI: <http://dx.doi.org/10.1145/2799250.2799290>

The context and history of driving simulators that we have reviewed portray them as tools for training or measurement. But for many researchers and automotive user experience professionals driving simulators are in fact development platforms on which they unleash their creativity and provide solutions to real world problems. In this paper we introduce Skyline, a driving simulator platform engineered in Intel Lab following principles of flexibility, integration and customization to enable rapid prototyping and testing of in-vehicle user experiences.

The rest of the paper is structured as follows. In section 2 we provide background on the design process in which Skyline as simulation platform was developed. In section 3 we describe the technical components of Skyline, including the physical and software components. In section 4 we detail the user experience development process using Skyline. Section 5 provides guidelines for user testing with Skyline, and Section 6 summarizes the conclusion and outlines future work.

DESIGN PROCESS

Intel has facilitated many Design Thinking workshops with Automakers. In a recent article about how Apple drives innovation [14], design thinking is described as “A systematic approach to problem solving that is user driven, possibility focused and iterative”. It is a process that was developed by leading design firms and is now taught at universities, with its methods leveraged by the most lauded innovators. Design thinking has gained recognition from many industries as a proven process that tackles complex problems from the point of view of the user, so that solutions identified are both novel and practical. Design thinking as a process is well documented, and Intel has produced several papers on how the use of this process has improved customer relationships. [15].

The most impactful outcome of a design workshop is the ability to build upon the concepts identified in the design thinking session conducted with a partner (e.g. an automaker or a research institution) to determine what is most meaningful to both companies to collaborate on as a research engagement. The concepts are ranked in priority and the most compelling is deconstructed into use cases. Those use cases are then rapidly prototyped in an experience design solution that can be evaluated with end users. Data analysis indicates how the usages should be optimized and then retested. This process has been completed with several partners, with use case features now currently in product development or widely available on the market. Because of the requirements created as an output of this process, the need for a driving simulator to operate as a development platform was identified and Skyline was commissioned.

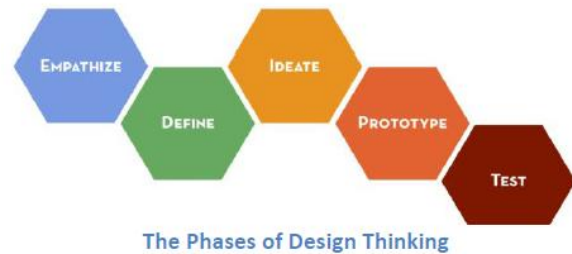


Figure 1. Phases of Design Thinking.

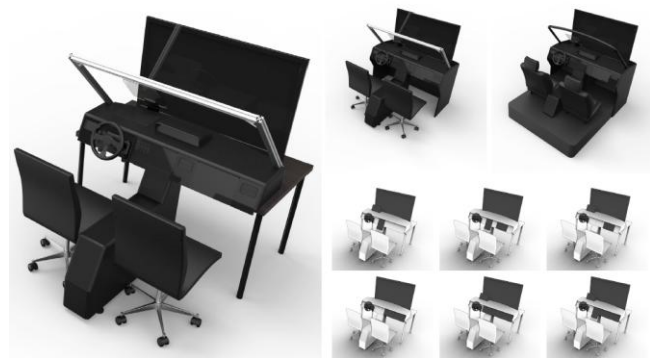


Figure 2. Skyline flexible physical configuration.

INTRODUCING SKYLINE

Skyline is a User Experience (UX) prototyping platform developed by Intel Labs to support design, research and learning about the personal vehicle experience. Skyline enables rapid development and visualization of in-vehicle concepts in simulated driving scenarios, thus becoming a powerful research tool for Intel Labs and its selected partners to enable iterative development of in-vehicle user interfaces through conducting user tests.

Skyline Physical configuration, “the buck”

The physical elements of Skyline, also known as “the Buck”, are built to enable customization of surfaces that can serve as both display and input areas. This flexible configuration is represented in figure 2.

Skyline basic components are:

- A 50” TV screen connected to a computer tower displays the virtual world in which the participant drives.
- A Logitech G27 Steering wheel and pedals connected to the computer allow the user to control the virtual vehicle.
- A polarized glass panel which reflects the content of a tablet in horizontal position onto the field of view of the driver, acting as translucent Head-Up Display (HUD).

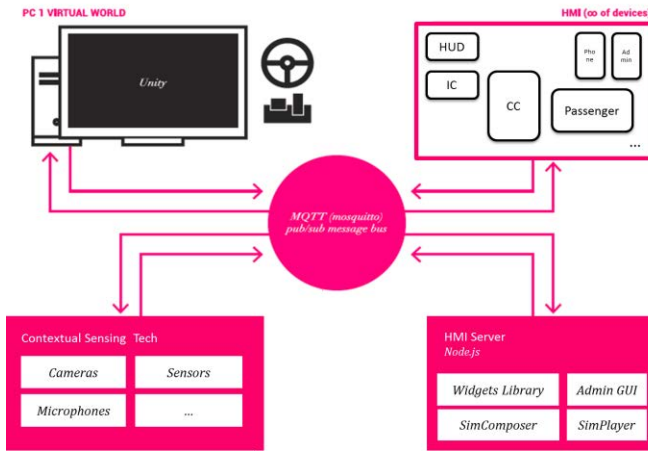


Figure 3. Skyline Software architecture.

- A variable number of Windows tablet PCs that act as in-vehicle cockpit displays. A typical configuration is centered on the driver and co-pilot experiences, HUD, Instrument Cluster, Center Console, and a Passenger display. The physical allocation of the displays along the dashboard is flexible and scalable. The selected number of tablets can be attached or detached to the frame, and non-display space can be occluded with snap-in panels. Screens can thus be rearranged to support and test a multitude of vehicle configurations and scenarios.
- A variable number of external consumer electronic devices (smartphones, tablets) can be linked to Skyline configuration and become part of the in-vehicle user experience. Typically the setup makes use of two smartphones, one acting as the driver's personal device and the other being a UX control from which a researcher performing user testing can flexibly trigger certain events or interactions in a "Wizard-of-Oz" (WOZ) style. This is especially useful in early stages of UX development.
- Leather vehicle seats for driver and passenger are the standard configuration of "the buck", however, the number of seats can be adjusted to portray a complete 5 or 7 seats vehicle.
- A wireless router enables the internal communication and synchronization of all the connected displays in Skyline.

Skyline Software Components

On the software side, Skyline is a real-time runtime environment that utilizes open source web technologies to connect each of the applications running on the display components. Figure 3 presents a diagram of the interconnected software components.

The tower PC runs a compiled executable of a 3D World Simulation developed using the Unity gaming framework [16]. Node.js provides a cross-platform runtime environment on which Web applications written on JavaScript exchange messages in response to events [17].



Figure 4. Skyline's virtual world scenery built in Unity.

In the background, the Message Queue Telemetry Transport (MQTT) messaging bus allows messages to be exchanged between the simulated driving world and the display devices [18]. Each of those devices runs a Web browser in full screen mode and is connected via Wi-Fi to the Skyline network. Next we will explain in detail how each of the components is constructed.

Front-end: Unity Virtual World

The virtual world in which Skyline users drive is a simulated city modeled using the city of San Francisco as a reference, and leveraging about one sixth of the total city map area. The Skyline world contains all the elements of a typical urban environment, including downtown area, suburban areas, and highways. See figure 4.

The virtual world is developed in Unity, a cross-platform gaming engine that can run on Websites, desktop platforms, consoles, and mobile devices [16]. The Skyline world can be modified using the Unity Integrated Development Engine (IDE) to add or customize 3D objects such as vehicles, buildings or terrain; as well as those objects' behaviors such as traffic events.

The basic behaviors needed for a driving simulator, such as road physics, traffic, and light traffic control are scripted into the compiled version of Skyline and can be used along three currently predefined routes through the city: a suburban route, a suburban to urban route, and a downtown route. In these cases, the user sees the virtual world on the screen from a behind-the-windshield point of view and drives the car along the predefined path, guided by a blue line in the center of the lane.

During the execution of the Unity program, the user will trigger events such as car telemetry (used by speedometers and map displays), street telemetry (used by stop light timers), and positional collision events. The positional collision events are transparent objects placed in the virtual world that, when intersected by the automobile, generate an MQTT event of a specified name.



Figure 5. Widget examples.

The Unity portion of Skyline can be compiled and run as a standalone application without need of the Unity IDE, but a Unity development license is needed to modify the layout, appearance or behavior of the Skyline world.

Front-end Applications (widgets)

The user interaction interface and elements are presented on tablets and smartphones configured into the Skyline setup. All the tablets and smartphones used in Skyline run on a Web browser in full screen mode. The browser presents ‘widgets’, which are laid out on the screens as required by the user interaction experience being tested. A cascade style sheet document (CSS) defines several ‘zones’ on each of the displays. The zone size and placement is defined in the CSS, and the binding between the zone name and the CSS is defined in an HMI configuration file.

Widgets are modular, scalable and reusable HMI assets that, once developed, can be deployed in other display areas or repurposed for different projects. As widgets are being applied in UX research projects, they might undergo configuration changes to adapt to a specific cockpit location, or the needs/requirements of specific users. Proper versioning and documentation of the configuration is thus required and is a fundamental part of the UX research process. Eventually a widget can develop derivative widgets applied for different use cases.

For example, the basic speedometer widget typically displayed in the instrument cluster can gradually be adapted for elders through rounds of user assessment to an HUD configuration that is less distracting. During the testing, font and color elements might be altered to bigger font size, with stronger color contrast as the best solution. The flexibility of CSS and JavaScript Web development allows makes it very simple to produce multiple variations of GUI designs, and the Skyline platform enables a flexible exploration of widget placement configuration across all the possible in-vehicle displays.

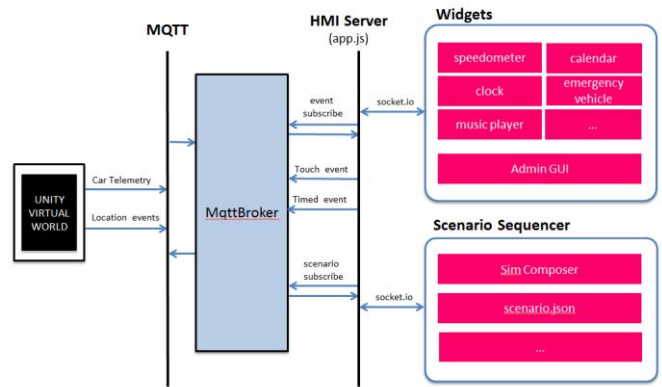


Figure 6. Skyline back-end communication diagram.

The Scenario Sequencer

The user experiences deployed on Skyline are driven by a ‘scenario sequencer’ that controls widget updates and ties the user interface interactions to driving progress through the virtual world. For instance, when the vehicle reaches a specific location in the world, some alert or notification may show up on the HUD.

When designing a scenario, the scenario sequencer presents an easy to use browser-based interface for updating the placement and appearance of widgets, and allows control over the interaction with world and user-triggered events.

Back-end Applications (MQTT and Node.js)

The two main back-end technologies that integrate the previously described front-end programs (Unity virtual World and HTML/JavaScript widgets) into a real-time communication framework are Node.js and MQTT:

- Node.js is an open source, cross-platform runtime environment for server and networking applications. It is mostly applied in real-time web applications such as Web-chats. Node.js applications are written in JavaScript, and can be run within the Node.js runtime on multiple operating systems (OS X, Microsoft Windows, Linux). It makes use of the Google V8 JavaScript engine to execute code [17]. It also contains a built-in library to allow applications to act as a Web server without need of HTTP or IIS Servers.
- MQTT (Message Queue Telemetry Transport) is an open source, publish-subscribe messaging protocol designed to connect remote locations with a small code footprint or under limited network bandwidth conditions. The publish-subscribe messaging pattern requires a message broker which is responsible for distributing messages to interested clients based on the topic of a message. It is an increasingly popular service for machine to machine (M2M) and Internet of Things (IOT) connectivity [18].

Figure 6 displays a diagram of the interconnection between Unity and the HMI using MQTT as message broker and Node.js as application server.

The browsers running on each of the HMI display tablets/smartphones are fed from a running node.js application server that, on launch, initiates the MQTT communications server, 'mqttBroker', which passes subscribed messages between the backbone MQTT bus and Node.js. Messages are passed to the HMI widgets via 'socket.io', a JavaScript library for real-time Web applications [19]. The reception of a socket.io message causes the widgets to dynamically update and changes the visual appearance on the display. In general, widgets can receive and send any MQTT message to and from the backbone bus.

DEVELOPMENT PROCESS USING SKYLINE

Skyline is a rapid ideation platform as well as a user testing assessment tool, and both aspects are key to its value proposition. Using this platform, automotive UX designers can focus on optimizing use case experiences through user testing, to define and document requirements before the expensive development phase of the product life-cycle begins. This can be a long process in the case of high fidelity assets in a driving environment. By using a rapid, flexible platform such as Skyline, a development team can explore with low investment and low risk different opportunities from the user's point of view, as well as validate and document the technical requirements that enable the optimized experience.

We previously referenced design thinking practices to be the starting point of the design development process. Whether the design team use this methodology or they opt for a different practice, it is crucial that at the beginning of the project some form of a design problem statement is formulated and a design solution is crafted to address the design problem based on foundational research. Designers can then develop design and interaction principles to be applied to that experience, such as the interaction principles described in figure 7 for a driver / passenger experience. These decisions will drive the iterative development of the necessary assets required to portray the solution experience.

Planning a Scenario

Using the widget and event triggers available in Skyline, (which can mean re-using, modifying or creating complete new widgets on the Skyline library), a UX designer can create a tailored experience or "scenario" along a specified driving path like the one described in figure 8.

Developing a scenario (or a driver user experience) in Skyline is accomplished by creating a sequence of "events". An event is a programmed trigger for a specific HMI action along the designated user driving path.

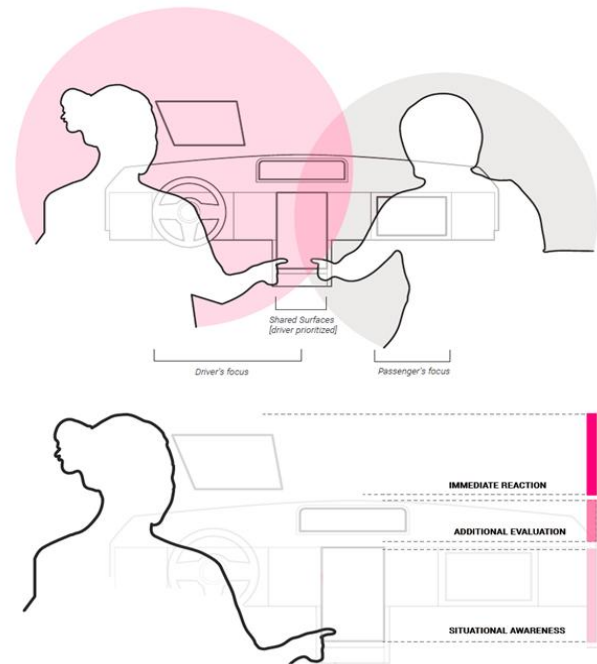


Figure 7. Examples of design guideline principles guiding Skyline prototype development.

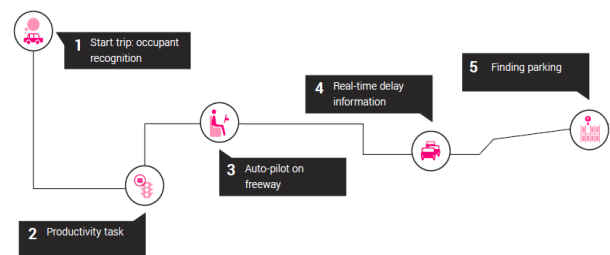


Figure 8. Planning a user experience in skyline.

Each event begins with a listener, which is a piece of code that listens for the trigger. When the event listener is called, it sets in motion all the actions within the event, which can include any specified combination of HMI widgets (visual, auditory or haptic) or elements in the virtual world (starting the car, engaging auto-pilot, or even controlling traffic signals). At the moment, Skyline can support 3 types of events:

- Timed events tie actions together in time so two actions are started at a specific time gap independently of what the user, or vehicle, is doing. For example, after starting the car, the seat-belt sign will be automatically displayed.
- Touch events are based on user input onto the HMI screens. The next actions will not happen until the user has triggered the event. For example, selecting a music source to play or setting the vehicle on parked mode.

- Location-based events are embedded in the Unity virtual world and are triggered by the vehicle driving through an exact location. For example, upon stopping at a red light, the display will be updated to show the count-down to green light.

In order to modify or create new location-based events, the UX developer needs to make use of the Unity IDE. However, timed events and Touch events can be programmed with text editors to create the JavaScript programs and CSS configuration files, or using Web development IDE like Aptana [20] or Intel's XDK [21].

Designing a Scenario

The “scenario sequencer” is a Web based tool that allows the design of a user driving experience using Skyline's existing widgets and driving paths. The tool requires first to load an existing driving path. Following that, ‘event blocks’ can be added. An event block can be composed of a minimum of one widget displayed in one HMI display, but usually is a number of widgets appearing in one or more displays, triggered by the reception of some MQTT event.

The scenario sequencer tool allows researchers to experiment during early design phases with information placement and UI designs that can even be created and saved during a user testing session. Once a configuration is set and tested, the UX researcher can save it with a specific name and launch it any other time.

Controlling a Scenario flow

During the early phases of UX development it is very important to capture as much qualitative feedback from users as possible. For example, a user might convey during user testing: “I like that the music options are always present here in the center console, but while I am driving I want to see them also on the HUD; this way I can change the source without taking my eyes of the road. Maybe from the steering wheel button or by voice commands”. Skyline was developed specifically to capture and rapid prototype based on this kind of feedback during longitudinal studies. However, the first time that a user faces the Skyline simulator s/he might have some difficulty absorbing the complete details of the experience while also concentrating on driving. In order for researchers to control this experience flow, an administrator controller view was built. The “Admin controller” provides direct control over all Skyline HMI states, allowing events to be triggered, held, or played back in a Wizard of Oz fashion. The admin view can be controlled from a computer connected to the Skyline network, or even from a small hand-held device like a smartphone as Figure 9 displays.

USER TESTING WITH SKYLINE

What makes Skyline so unique is that the rapid prototyping platform allows users to perceive the test stimulus in the

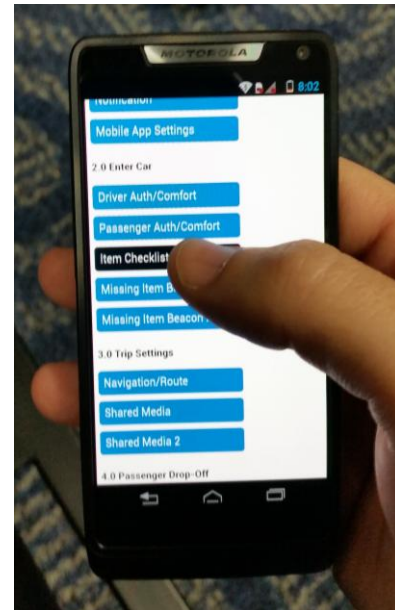


Figure 9. Controlling the scenario in WOZ testing in skyline.



Figure 10. Test participant in parked state answers a questionnaire.

context of daily driving. The Skyline driving world was built to resemble daily driving activities of a majority of the driving population. As such, the driving experience resembles commutes from home to the office, taking a highway, or going downtown. Skyline is not purposed to perform driver performance measurement tasks such as lane changes, object avoidance, or follow the front car; although such measures could be implemented in the future. Skyline is designed for eliciting qualitative feedback based on a realistic driver experience.

The tested assets (widgets and interactions) can be as low fidelity as glorified wireframes or as high fidelity as appearing in-vehicle production ready, depending on the test objective. As the assets are essentially widgets that can be reassigned placement based on user feedback, it logically follows that the assets themselves can be rapidly altered.

Throughout the scenario experiences, experiential trends or themes emerge from data analysis process, design and technical development. These themes create opportunity hypotheses that require further evaluation. These hypotheses ideally culminate in a concept to prototype that can be rapidly tested and ideated based on feedback gathered from user participants.

These prototype outcomes influence and inform each other as well as the initial hypothesis. The data is always the baseline, and being true to the data (whether it is favorable or not), is an imperative.

User Experience Assessment or user testing needs to be planned on specific target groups and needs to provide sufficient data. Once there is enough data to collectively agree on a perspective, the prototype findings, new methodologies, or outcomes can then be shared. However, several rounds of user assessment might be required to validate a design concept or a use-case in a mature form.

The testing methodology using the Skyline platform is primarily qualitative assessment probes, using medium to high fidelity test assets, WOZ or developed interactions, facilitator observation, and interview-style inquiry. These sessions are usually recorded with a field of view to encompass both the facilitator and the participant, but future roadmaps include the integration of innovative technologies such as cameras and motion sensors.

CONCLUSION & FUTURE WORK

In this paper we introduced Skyline, a rapid user experience prototyping platform developed by Intel Labs to address automotive user interface and researcher and designer in-vehicle application needs. We showed how Skyline's physical configuration has the advantage of being flexible and highly customizable. Skyline allows dynamic placement of a variable number of in-vehicle displays allowing tailoring of the simulator cockpit for the driver only, or full vehicle experiences. Furthermore, Skyline also enables easy inclusion of consumer electronics into the in-vehicle experience.

On the software side we highlighted the smooth adoption of Skyline as a software development platform, as it enables researchers to develop interface concepts with widely extended standard Web technologies such as JavaScript and CSS. Using these to provide the front-end visual Skyline also makes use of standard real-time communication technologies such as Node.js and MQTT to deploy these design assets in the form of fully functional widgets in a driving simulator environment. We also explained how non-technical UX researchers can utilize the existing library of widget assets to develop a complete daily driving integrated experience in a 3D world. The paper also explained how user experience assessment can be conducted with Skyline to collect qualitative feedback to be utilized in iterative design processes at different fidelity levels, from Wizard of Oz to high fidelity.

The amount of data gathered during this kind of iterative assessment can be invaluable for automotive OEMs where many times, experience design teams see their decisions on future design concepts challenged by engineering and upper management, or where creativity is constrained to proprietary SDKs with limited functionality. We believe Skyline is a flexible easily adoptable tool which can be employed successfully during iterative design phases to provide highly valuable production-ready HMI assets as part of future user experiences. Furthermore it will allow design decisions (which have a tremendous economic impact on engineering and product development), to be thoroughly documented as user-centric.

For researchers, Skyline is a highly effective tool to devise exploratory research hypotheses and rapidly develop experience solutions in realistic driver environments. The data gathered through this process has internally proven to be of tremendous value, and has driven multiple publications. The documented process of iterative development in Skyline could provide a UX Team with enough quantitative and qualitative data to justify their decisions, whether in a project design phase or during research and development. Another potential group that could benefit from the adoption of Skyline as a development platform are automotive production engineering teams, who would then have a prioritized feature list of validated user experiences on which they can focus on integration, reliability, testing and production costs, as well as have funded data for developing appropriate business plans.

Finally, a high fidelity prototype in Skyline has multiple values. High fidelity scenarios in Skyline are a powerful communication tool for automotive design teams to take upwards to management, or for the general public. A Skyline demo presents high quality realistic depictions of future concept cars technology. Skyline can also be easily mobilized for Autoshow, conferences, conventions or expositions to help expose novel automotive design concepts to the public.

Future work

Future work on the Skyline platform aims at the integration of contextually relevant data sources and sensors. Integrating new technologies such as depth cameras into the development platform would enable easy access to RGB and depth data, as well as existing libraries for facial, posture, and gesture recognitions which could then be easily integrated into the vehicle user experience. Other extensions could look at the integration of contextual sensor data, enabling researchers to obtain and analyze quantitative data to complement the qualitative data analysis described in this paper for a holistic data set.

REFERENCES

1. Fisher, D. R. 2011. Handbook of Driving Simulation for Engineering, Medicine, and Psychology. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group.

2. Yoshimoto, K. a. 2008. The History of Research and Development of Driving Simulators in Japan. *Journal of Mechanical Systems for Transportation and Logistics*, 1, 159-169.
3. Alexander, A. B. 2005. From Gaming to Training: A Review of Studies on Fidelity, Immersion, Presence, and Buy-in and Their Effects on Transfer in PC-Based Simulations and Games. *DARWARS Training Impact*, 2-14.
4. Backlund, P. 2009. SIDH: A Game-Based Architecture for a Training Simulator. *International Journal of Computer Games Technology*, 1-9.
5. Mueller, J. M. 2014. Driving Simulator and Scenario Effects on Driver Response. *Proceedings of the 2014 Industrial and Systems Engineering Research Conference*. Ontario: IEEE, 507-513
6. Weinberg, G. A. 2009. Developing a Low-Cost Driving Simulator for the Evaluation of In-Vehicle Technologies. *First International Conference on Automotive User Interfaces and Interactive Vehicular Applications*. Essen: ACM. 51-54
7. Boyle, L. a. 2010. Using Driving Simulators to Assess Driving Safety. *Elsevier Accident Analysis and Prevention*, 785-787.
8. Weir, D. a. 1995. A Survey of Mid-Level Driving Simulators. *International Congress and Exposition* (pp. 53-55). Detroit, MI: SAE International.
9. Chan, E. P. 2010. Are driving simulators effective tools for evaluating novice drivers' hazard anticipation, speed management, and attention maintenance skills? *Elsevier Transportation Research Part F: Traffic Psychology and Behaviour*, 343-353.
10. Mattes, S. 2003. The Lane Change Task as a tool for driver distraction evaluation. *IHRA-ITS Workshop on Driving Simulator Scenarios*. Dearborn, Michigan: NHTSA.
11. Math, R. M. 2012. OpenDS: a new open-source driving simulator for research. *Adjunct proceedings of the 4th International Conference in Automotive User Interfaces and Interactive Vehicular Applications*. Portsmouth, NH, USA: ACM. 7-8
12. Green, P. 2012. Using standards to improve the replicability and applicability of driver interface research. *4th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*. Portsmouth, NH: ACM 15-22
13. Green, P. 2013. Standard Definitions for driving measures and statistics: overview and status of recommended practices J2944. *5th International Conference on Automotive User Interfaces and Interactive Vehicular Applications*. Eindhoven: ACM. 184-191
14. Liedtka, J. Forbes 2012 Retrieved March 28, 2015 from <http://www.forbes.com/sites/darden/2012/05/01/designing-for-growth-apple-does-it-so-can-you/>
15. Roger Chandler, 2014, *Adventures in Design Thinking*, February 24, 2015, Intel Library
16. Unity Technologies. 2015. Unity3D. Retrieved February 24, 2015, from <http://unity3d.com/>
17. Joyent, Inc. 2015. Nodejs.org. Retrieved February 27, 2015, from <http://nodejs.org/>
18. MQTT.org. 2015. MQTT.org. Retrieved February 27, 2015, from <http://mqtt.org/>
19. socket.io. 2015. Retrieved February 27, 2015, from Socket.IO: <http://socket.io/>
20. Appcelerator, Inc. 2015. Aptana.com. Retrieved February 27, 2015, from <http://www.aptana.com/index.html>
21. Intel Corporation. 2015. Intel Developer Zone HTML5. Retrieved February 02, 2015, from Intel XDK: <https://software.intel.com/en-us/html5/tools>